Exploring the Impact of Community Sentiment and Programming Language Popularity on Open Source Software Success

AP Research

21 April 2025

Word Count: 5,427

Introduction and significance

Open-source software (OSS) is any software that has its code publicly viewable and modifiable. Unlike proprietary software, OSS enables developers from all over the world, and more importantly across companies, to collaborate on the same software project. OSS projects such as Git, Linux, and TensorFlow, can each demonstrate the great impact an OSS project can have. Linux, for example, powers the majority of servers running the internet, and even Microsoft Azure, a major cloud provider, switched to Linux in 2015 (Finley, Klint). TensorFlow, especially in the current artificial intelligence boom, proved instrumental for machine learning and artificial intelligence research. Furthermore, OSS is crucial to many businesses' operations as it enables cost-effective software adoption due to its reusability by potentially thousands of companies (Chesbrough, H). According to Chesbrough H., 47% of business leaders agreed that it would be twice as expensive to build an in-house solution as opposed to using OSS. Consequently, Hoffmann M. estimates that developing all OSS in-house would cost around $8.8 trillion. Another estimation by Hoffman M. emphasized that software development without OSS would be as much as 3.5 times as expensive. Despite its significant economic impact, OSS often struggles to achieve long-term sustainability due to its inherent characteristics: lenient licensing, community-driven development, and the risk of developer burnout. While achieving sustainability, especially in the long term, requires careful balancing of resources, one of the most important aspects of OSS is the community behind it. A steady flow of interest and engagement from the community can ensure that the software is maintained and developed. To support OSS, this paper therefore aims to explore trends in successful, popular, OSS projects on GitHub, providing insight into what helps OSS get people's attention and grow its community. In particular, this paper focuses on analyzing the impact that the size of the community of the primary language of the repository has on the success of the repository. Additionally, this paper also assesses the positivity of the

community interactions and their correlation to the repository success, providing further insight into the importance of community interactions in OSS.

This paper will be collecting data from GitHub only because it is by far the largest and most common developer platform for distributing code and OSS. Each project on GitHub, although sometimes split up into several, is a repository. With over 100 million public repositories, GitHub is used by organizations and individuals alike, providing a broad population of OSS to sample for this paper (GitHub, “Search stars:>=0”). While alternatives exist such as GitLab and BitBucket, they aren’t as mainstream as GitHub. Furthermore, their APIs are more limited compared to GitHub’s which would have reduced the collectable data. For these reasons, I chose to focus on GitHub solely. The generalizability of this data will be treated as such.

The success of GitHub repositories will be determined by the number of stars it has. A star is very similar to a “like” on a social media app; it serves as an indicator of a project's popularity, reflecting its likeability and user engagement, particularly engagement from potential donors and contributors. As will be discussed in greater detail in the literary review, many peer papers also use the star count to quantify the popularity of a repository. This paper will therefore use the number of stars as an indicator of a project’s success.

The sentiment analysis of GitHub comments will be done using SentiStrength-SE (Islam, Md Rakibul). Specifically, SentiStrength-SE will be used to analyze comments on issues, pull requests, and commits on GitHub. The primary reason for choosing SentiStrength-SE is as compared to other tools such as VADER (which can be found at github.com/cjhutto/vaderSentiment), SentiStrength-SE is fine-tuned for software engineering and highly technical discussions making it domain-specific. Additionally, SentiStrength-SE is widely used in software engineering research which enables for more in-depth and accurate comparisons to other research papers (Assavakamhaenghan, Noppadol). SentiStrength-SE

has several key advantages over other domain-specific sentiment analysis tools as well. Since SentiStrength-SE uses lexicon-based techniques for sentiment analysis, which rely on a predefined dictionary of words with assigned sentiment scores rather than complex machine learning models, it is significantly faster than other more computationally expensive tools such as SentiSE. Another tool considered, SentiCR (Ahmed, Toufique), wasn't used because it was specifically trained for code review comments, but this paper primarily analyses discussion comments. For the same reasons as SentiSE (which can be found at github.com/amiangshu/SentiSE), SentiCR is also significantly slower than SentiStrength-SE. Since this paper will be analyzing millions of comments, SentiStrengthSE is the best option despite the lower accuracy.

The data was collected using the official GitHub API for research purposes ensuring that GitHub's policies and terms of service were followed. In total, 221 thousand repositories were collected, from which 105 thousand repositories were analyzed to determine the effect of different primary programming languages on the success of OSS repositories. Additionally, from the 105 thousand repositories, 8 thousand repositories had comments collected and analyzed for sentiment. In total, 3.3 million comments were collected of which 3 million were analyzed to determine the effects of average positive or negative sentiment for a repository on the success of a GitHub repository.

Literary review

Defining Success

For further analysis, it is crucial to define how the success of a repository will be measured. This paper will define the success of a repository by its popularity. However, popularity is also very vague and difficult to quantify. Related works in this field use different methods of measuring popularity such as relying only on the star count, but other options include weighted sum of stars and forks. After a thorough review, this paper will be only

using stars because it provides the most reliable indication of popularity (Borges, Hudson). Moreover, it is the most frequently used method in related works as well, most notably by Milicin.

Community interactions

The impact that a community can have on the success of a repository is profound as it is crucial in not only attracting but keeping contributors. Assavakamhaenghan investigated the role of sentiment in influencing newcomers' likelihood of contributing again to a project, an instrumental component of achieving sustainability in OSS. Assavakamhaenghan's research, which used a combination of SentiStrength-SE (Islam, Md Rakibul), Toxicity (Sarker, Jaydeb), and EmoTxt (Calefato, Fabio, and et al.) and analyzed over 2 million first contributions, found no correlation between sentiment and future contributions. However, they did find that first responses to first contributions were more likely to be positive as well as established a positive correlation between the number of responses and the likelihood of contributing again, showcasing the importance of an active community. My research uses a similar, but wider method for sampling discussions, providing a broader view of the effects of sentiment; I analyzed the sentiment of all discussions as opposed to only the first interaction for the OSS repository. However, I only focused on SentiStrength-SE as compared to the wider range of emotions analyzed by Assavakamhaenghan.

Another researcher, Szatmáry P. studied the impact that a lack of documentation has on a repository. They found that a lack of documentation has no harmful effects on the repository because newcomers, who might have benefited from documentation, can get help from community members instead. This once again highlights how crucial a community is for an OSS project. However, the paper did not further investigate the types of interaction that are involved in helping newcomers. My paper builds on top of Szatmáry P.'s work by

investigating the sentimentality of the discussions that happen between the newcomers and the community members and its impact on the star count.

Finally, Shimada N. has established that developers with a greater social media presence and developers who are more public about their plans generally receive greater donations. My research's focus on the community can support similar research but for the OSS projects themselves. While my research doesn't directly address funding, finding aspects of OSS that contribute to a project's popularity can highlight what helps grow a community of potential donors. The importance of a community is further highlighted by the fact that on Open Collective, a platform for donating to OSS, OSS receives overall more donations from individuals rather than the often larger but fewer corporate donations (Zhou, Jiayuan).

Programming language

As another component to determine the effect that a community can have on a repository, this paper will also be analyzing the impact that the community size of the primary language used in the repository on the repository's success, star count. Andrei Milicin has determined that programming languages do, in fact, have an impact on the number of stars a repository has. However, no cause was attributed to impact. My research paper investigates a potential one: the number of developers using the programming language. Moreover, my research uses a significantly larger dataset of 105 thousand repositories as opposed to the 2,690 repositories used by Milicin. It is important to note, however, that Milicin focused on all GitHub repositories whereas my research focuses only on OSS GitHub repositories.

Shimada N. has also done similar research, but for developers on GitHub. Specifically, Shimada N. found that developers contributing with popular languages (JavaScript, Python, PHP) received more donations. My paper aims to help determine whether the same relationship applies to repositories on GitHub as well by doing a correlational analysis

between the popularity of the programming language and the success of the repository in terms of star count.

Furthermore, several other papers have used GitHub's API to collect data and analyze language use and patterns in repositories. For example, Sanatinia and Noubir used GitHub's API to determine the top 10 most common primary programming languages, in terms of the number of repositories, on GitHub in 2016: JavaScript, Ruby, Python, Objective-C, Java, PHP, CSS, C, C++, and Go. However, OSS trends rapidly change as Altherwi found the top 10 programming languages on GitHub in 2021 to be JavaScript, Java, Python, Go, C++, PHP, C, Objective-C, Swift, TypeScript, HTML, Ruby, C#, Shell, and CSS, in order. Going even further, Milicin found that in 2024 the top 10 were JavaScript, Python, TypeScript, Go, Java, C++, Rust, C, Shell, and Jupyter Notebook, noting that Jupyter Notebook is "synonymous to Python and Machine Learning" (Milicin, Andrei-Radu). As highlighted by the differences in the rankings, it is crucial for my research to establish an up-to-date and reliable way to measure a language's popularity. As such, this paper will be using the most recent developer surveys to gauge the popularity of programming languages. There are two large-scale developer surveys: Stack Overflow 2024 Developer Survey and JetBrains Developer Ecosystem Report 2024 (Stack Overflow; JetBrains). However, with 65,437 responses from around the world, Stack Overflow's survey had 3 times as many responses as JetBrain's survey. Furthermore, StackOverflow's top ten (JavaScript, HTML/CSS, Python, SQL, TypeScript, Bash/Shell, Java, C#, C++, and C) is more in line with Milicin's most recent findings than JetBrain's top ten. Further comparisons and justifications will be made in the method section of this paper.

Gap

My gap consists of 3 parts. First, my focus on the average sentiment of discussions for OSS repositories on GitHub has never been research. Similar works, such as

Assavakamhaenghan and Szatmáry P., have explored the impact of sentiment and a community on OSS, however, neither has taken a broad, overall, view of the effect that sentimentality of discussions have on the OSS. Second, Milicin has found that programming languages affect the popularity of a repository, however, no reason for this was investigated or proposed. My research investigates a potential one, the popularity of the programming language itself. It is important to note that Milicin analyzed all GitHub repositories while I focused only on OSS GitHub repositories adding to my gap. Finally, technology, especially software development, is a rapidly changing field as highlighted by the quickly changing top 10 most common programming languages on GitHub (Sanatinia, Amirali, 2016; Altherwi, Muna, 2021; Milicin, Andrei-Radu, 2024). For this reason, my method also focuses on collecting the most up-to-date data possible.

Method

My method can be split into 5 parts: collecting repository data from GitHub, filtering repositories, collecting comments from repositories, filtering comments, and analyzing the sentimentality of comments. The full codebase I programmed and used to collect, filter, and analyze data is publicly viewable at https://github.com/BoogeyMan24/ap-research.

<u>Collecting repositories</u>

My data consists of all 221 thousand (221,672) public GitHub repositories with at least 200 stars. This data was collected in 4 hours on January 20th. I chose to only analyze repositories with more than 200 stars for several reasons. The primary reason is that due to GitHub API limitations, I can't reliably request repositories that are below 200 stars. This is because GitHub limits the number of repositories returned per search to one thousand (see Figure 1). I was able to get around this issue for highly starred repositories by collecting the data in ranges of stars instead; rather than requesting all repositories above 200 stars which

would have far surpassed the limit, I searched for ranges of stars (e.g. from 2,000 to 2,070 stars) that would result in less than one thousand repositories found (see Figure 2).

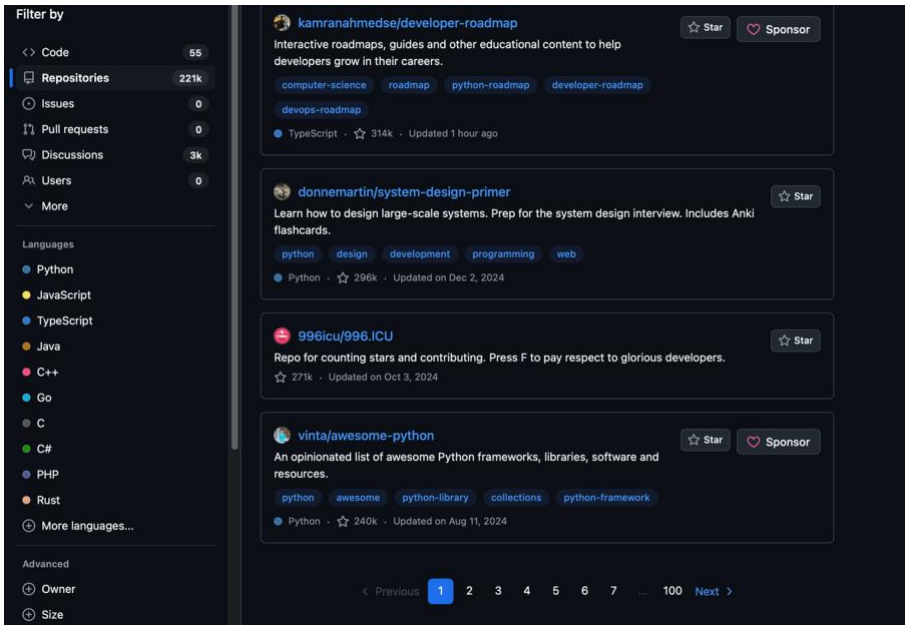

Figure 1. Source: *GitHub*. "Search stars:>=0." GitHub, github.com/search?q=stars%3A%3E%3D0&type=repositories. Accessed Apr. 2025.

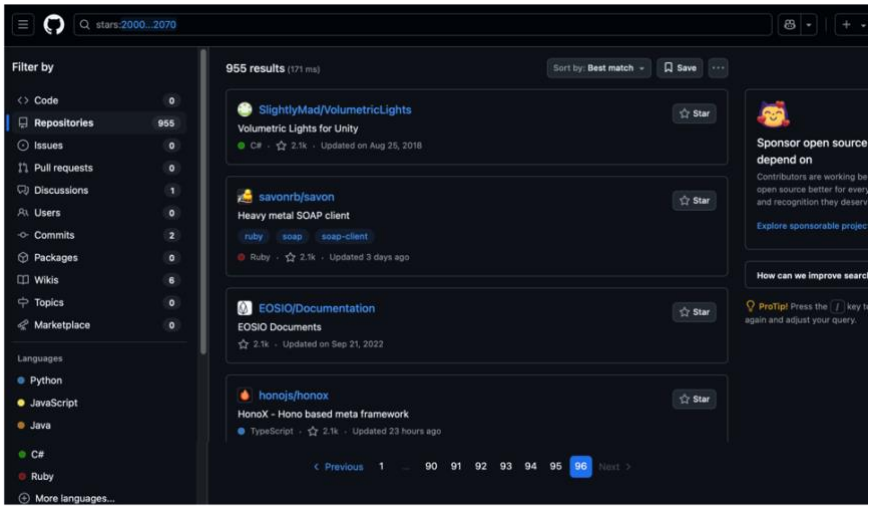

Figure 2. Source: *GitHub*. "Search stars:2000..2070." GitHub, github.com/search?q=stars%3A2000..2070&type=repositories. Accessed Apr. 2025.

Furthermore, the 200 stars boundary guaranteed that I would not get a star range such as 191 to 191 which returns over 1000 repositories resulting in inaccessible repositories. Additionally, the boundary helped ensure that all data collected were serious OSS projects as

opposed to hobby projects. To collect the data, I programmed a script to automate the process (see Appendix A). For each repository, I collected the following data: name, description, URL, size, stars, forks, primary programming language, and topic tags.

Filtering repositories

Not all data was analyzable, however. Many high-starred repositories aren't OSS projects, but rather a list of useful resources. For example, the second most starred repository on GitHub is "free-programming-books" which simply lists programming books. To filter out non-OSS repositories, I used the topic tags associated with a repository to construct a list of blacklisted topic tags. To construct the list of blacklisted words, I manually found a random sample of 100 non-OSS projects and picked the topic tags that were common among them all. The list of blacklisted words was as follows: interview, education, career, certification, community, roadmap, tutorial, curated, list, awesome, tips, intern, resource, job, hunting, trick, pdf, book, template, and documentation. It is important to note that any variation of these words was also excluded as the blacklisted phrase only needs to be part of the topic tag. For example, "interview-prep" is blacklisted because it contains "interview." This also applies to their plural forms. To ensure the representativeness of the data, any project without any topic tags was also excluded as I could not reliably verify it was an OSS project. It is possible that the list of blacklisted words mistakenly excluded OSS projects, but my sample size is so large, its effect will be insignificant in the final analysis. This filtering process generated the dataset of 105 thousand GitHub repositories which was used to do the correlational analysis between the star count and the popularity of the programming language.

However, for the sentiment analysis using SentiStrength-SE, I had to ensure that the OSS repositories were also primarily in English. It is impossible to determine the language of the repository from its name because GitHub enforces the English alphabet on the name of

the project. I also could not have filtered the repositories' descriptions for English-only characters because many contained emojis and other special characters that would have resulted in false positives. Instead, I ran the description of the repository project through several public software packages for language detection. Specifically, I attempted language detection with the "languagedetect," "eld," "langcheck," and "is-english" NPM packages, but none were accurate enough, especially with logographic languages. In the end, the most precise method was to check if the description contained any commonly used Chinese, Japanese, Korean, or Cyrillic character codes. This was done with this regex \u0400-\u04FF\u3400-\u9FBF 一-鉞ぁ-んァ-ン. This method was the most effective because the most popular languages for OSS GitHub repositories in the dataset were English, Chinese, Korean, Japanese, and Russian. This method's accuracy with the 4 most used languages after English significantly outweighs its ignorance of other languages, especially when compared to the inaccuracy of the language detection libraries. Ultimately, this process resulted in 95,119 English-only OSS GitHub repositories. This data will be used to collect the comments which will then be analyzed for sentiment. By averaging the comments, a single sentiment score will be given to the repository for further analysis.

Comments and filtering

From the 95,119 English-only OSS GitHub repositories, around 13,264 repositories were randomly selected to collect comments from. The comments from the 13 thousand repositories were collected in a week, from 17 February to 24 February. I planned to collect comments from at least 8 thousand repositories which would give me a large sample while ensuring the independence condition could still be met; however, not all repositories will have analyzable comment data, so an extra 5 thousand was collected as cushioning. Furthermore, I aimed to collect 500 comments from each repository to establish a margin of error (for a 95% confidence level) of at most 0.2. This sample size was estimated using 1 as

the expected standard deviation, which, as I will show, is a generous overestimate. Because SentiStrength-SE gives sentiment scores from 1 to 5 and -5 to -1, the mean is guaranteed to be between these values and never produces any outliers. The highest sample standard deviation possible is therefore 2. However, assuming that the score is random and not skewed yields a sample standard deviation of 1 on average. The comment data is nearly guaranteed to be at least slightly skewed to the right (toward neutral sentiment) leading to a sample standard deviation of less than 1. This leads to an estimated sample size of 100 comments required to establish a margin of error of 0.2 with a standard deviation of 1 at 95% confidence. I collected a significantly larger sample size to guarantee that this threshold will be met even in scenarios where a majority of comments are unusable due to being bot-generated or containing a majority of non-English text. In the end, from the 13 thousand, 8 thousand (8,055) had at least 100 analyzable comments with an average of 356.833 comments per repository.

The process for collecting comments is as follows. First, the script I programmed (see Appendix B) randomly picks a repository from the English-only dataset and makes an initial request to the GitHub API to determine the total number of comments for that repository. If the number of comments in total is less than 500, all available comments are collected. On the other hand, if the number of comments is more than 500, 500 are randomly selected from the total to be collected ensuring random sampling. Only once all the comments are collected from a repository are they saved to a file to guarantee that even in the case of an error, power cut, or connectivity issue, no partial data is saved. The repository is then saved to a file that keeps track of the repositories already completed and the number of comments collected. In total, 3,298,140 comments were collected of which 3,052,278 were human-made. Finally, the 3 million comments were analyzed as part of the 8 thousand filtered repositories. The

following data was collected for each comment: repository name, comment ID, user association, user type (either “User” or “Bot” as set by GitHub), and body text.

Analyzing sentimentality

For the analysis, it is important to understand how SentiStrength-SE analyzes sentiment. SentiStrength-SE uses a lexicon-based analysis which means it uses a predefined list of words, abbreviations, and phrases with a weighted score for their positive and negative sentiment. This list is domain-specific meaning it contains even highly technical phrases. SentiStrength-SE is then able to cross-reference the list to the comment to generate both a positivity score (from 1 to 5) and a negativity score (from -1 to -5). This means that every comment has both a positive and a negative score where either a 1 or -1 expresses neutrality and any increment expresses stronger positive or negative sentiment. The fact that SentiStrength-SE uses a lexicon-based analysis makes it significantly more efficient at the cost of potentially missing key phrases that can result in lower accuracy. Due to the large number of comments, I analyzed the sentiment in batches of 100 thousand comments.

Popularity of programming languages

To quantify the popularity of programming languages, I chose to use developer surveys that asked: What programming languages have you extensively used in the last 12 months? The percentage of respondents who used that language will be used to quantify the popularity. Specifically, two large developer surveys asked this question and were considered for the analysis: Stack Overflow Developer Survey 2024 and JetBrains Developer Survey 2024. After careful consideration, I used the Stack Overflow survey rather than the JetBrains survey because it not only had a greater number of responses, Stack Overflow had 64 thousand while JetBrains had 23 thousand but also was more in line with Milicin’s findings for the top 10 most popular programming languages. Additionally, the most important reason for choosing Stack Overflow’s survey was that the JetBrains survey had a bias towards Java.

JetBrains has strong ties to the Java community due to it being the creator of the most popular integrated development environment (IDE) for Java: IntelliJ IDEA CE. This bias is further illustrated in Table 1 where Java has the greatest difference in popularity between the two surveys, more than double that of the second.

Programming Language	JetBrains	Stack Overflow	Absolute Difference
Java	46.0%	30.3%	15.7%
Python	57.0%	51.0%	6.0%
C#	22.0%	27.1%	5.1%
Kotlin	14.0%	9.4%	4.6%
Go	18.0%	13.5%	4.5%
SQL	48.0%	51.0%	3.0%
C	18.0%	20.3%	2.3%
Bash/Shell	36.0%	33.9%	2.1%
C++	25.0%	23.0%	2.0%
HTML/CSS	51.0%	52.9%	1.9%
TypeScript	37.0%	38.5%	1.5%
JavaScript	61.0%	62.3%	1.3%
PHP	17.0%	18.2%	1.2%
Average	34.6%	33.2%	3.9%
Average w/o Java-based	35.5%	35.6%	2.8%

Table 1. Sources: Data compiled by the author from Stack Overflow, “2024 Stack Overflow Developer Survey” and JetBrains, “Software Developers Statistics 2024.”

Stack Overflow’s survey results that will be used for the analysis can be seen in Figure 3.

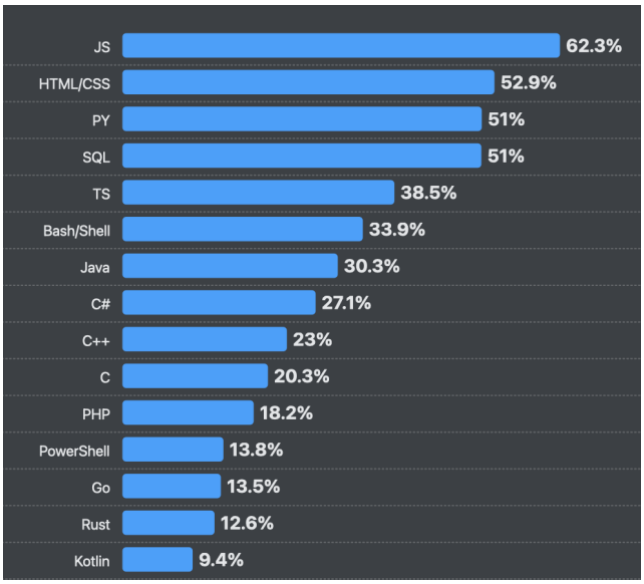

Figure 3. Source: Stack Overflow, “2024 Stack Overflow Developer Survey.”

Analysis

Hypotheses

To help guide us through the analysis, I will be using these research questions and hypotheses with $\alpha < 0.01$:

1. RQ1: Is there a correlation between the sentimentality of discussions on a GitHub repository to its success (star count)?
    - H_o (Null Hypothesis): There is no significant correlation between average sentiment scores and repository star count.
    - H_a (Alternative Hypothesis): There is a significant correlation between average sentiment scores and repository star count.
2. RQ2: Is the popularity of a programming language correlated to the success of a GitHub repository (star count)?
    - H_o (Null Hypothesis): There is no significant correlation between the primary programming language's popularity and repository count.
    - H_a (Alternative Hypothesis): There is a significant correlation between the primary programming language's popularity and repository star count.

Research question 1

To determine the effect that positive and negative sentimentality of community interactions have on the success of a repository, I calculated the average positivity and negativity score for each repository. To construct the confidence interval for the average positive and negative sentiment of discussions on OSS GitHub repositories, I ensured all conditions for inference were met:

- Random sampling: the OSS projects were randomly selected from the 95 thousand English-only OSS repositories. The generalizability of the data will be treated as such.

- Independence: the analyzed sample size of 8 thousand repositories is far less than 10% of the total GitHub OSS projects (which I estimated at least 95 thousand).
- Normal distribution: due to the large sample size, the central limit theorem guarantees that the sample approximates a normal distribution. For extra assurance, Figure 3a and Figure 3b were constructed where both positive and negative sentimentality distributions follow a bell-shaped normal distribution.

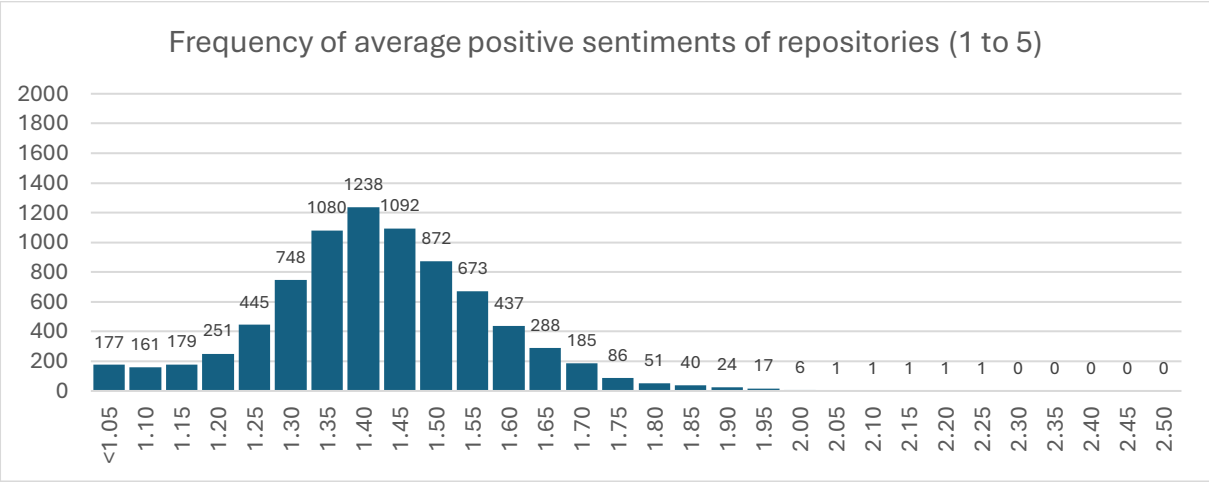

Figure 3a. Source: author. "Distribution of average positive sentiments of repositories." Apr. 2025.

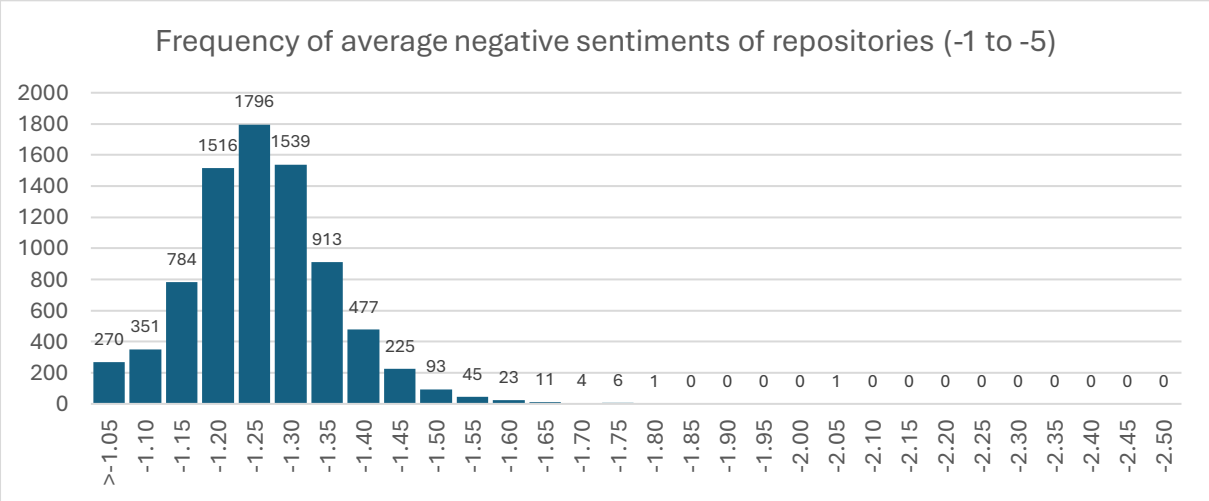

Figure 3b. Source: author. "Distribution of average negative sentiments of repositories." Apr. 2025.

To analyze the average sentimentality, I constructed a confidence interval for both positive and negative sentiments which can be seen in Table 2. Both positive and negative null hypotheses were rejected ($p < 0.01$) meaning at a 99% confidence the true average

positive sentiment is between 1.3908 and 1.3975 and the average negative sentiment is between -1.2324 and -1.2367.

	Positive Sentiment	Negative Sentiment
Average	1.39415	-1.23455
Lower bound	1.3908	-1.2324
Upper bound	1.3975	-1.2367
P-value	0.00337	0.00213
Null hypothesis	Rejected	Rejected

Table 2. Source: author. "Confidence intervals of mean sentiment scores with p-values." Apr. 2025.

Because there is no overlap in the positive and negative intervals, I can also conclude that comments tend to be overall more positive than negative. This aligns with Assavakamhaenghan's findings, where the first interaction (i.e., the first comment) on a newcomer's contribution was also generally more positive. Interestingly, this result is somewhat unexpected, given that English tends to be considered a more negative language. Moreover, the difference suggests that Assavakamhaenghan findings about the positivity of first interactions can be applied to comments more broadly. These findings also support Szatmáry P.'s reasoning for why a lack of documentation has little effect on newcomers: even without documentation, newcomers can ask community members for help and the overall greater positivity of comments makes this support more approachable.

For the correlational analysis of the average sentiments to star count, I will be calculating the Pearson correlation coefficients for the average positive and average negative sentiments to the star count.

Figure 4a shows a scatterplot of repositories with the average positive sentiment on the x-axis and the star count on the y-axis. Additionally, the correlation coefficient was calculated to be a weak negative correlation ($r = -0.0557$).

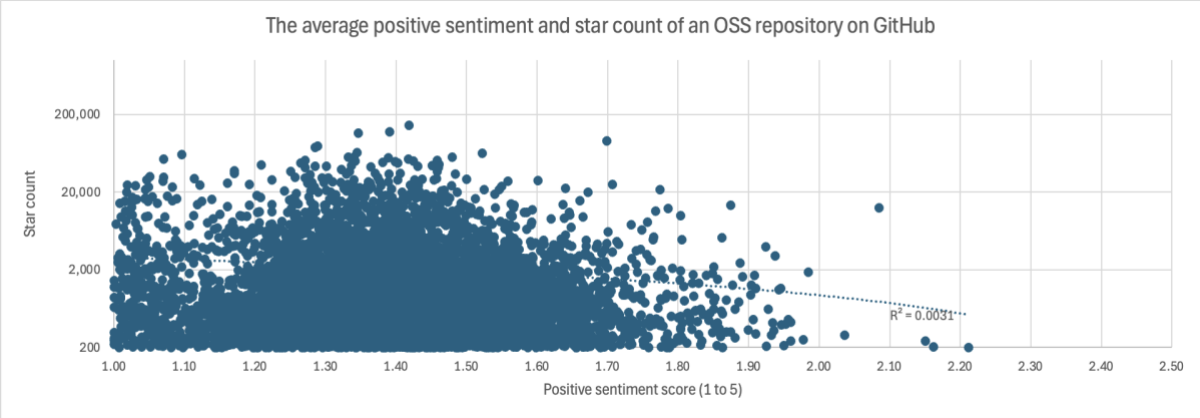

Figure 4a. Source: author. "The average positive sentiment and star count of an OSS repository." Apr. 2025.

This suggests that as the average positive sentiment for a repository increases the star count of the repository decreases. The p-value for the correlation is 0 making it extremely unlikely to have happened by chance. The null hypothesis is therefore rejected, establishing a weak negative correlation between the average positive sentiment of comments on a repository and the star count of the repository. However, the small p-value is more of a statement of my large sample size than the actual correlation. In reality, a correlation coefficient of -0.0557 is too small to be useful or practical in any way.

Figure 5b shows a scatterplot of the repositories with the average negative sentiment on the x-axis and the star count on the y-axis. The calculated correlation coefficient was calculated to be also weak and negative (r = -0.03).

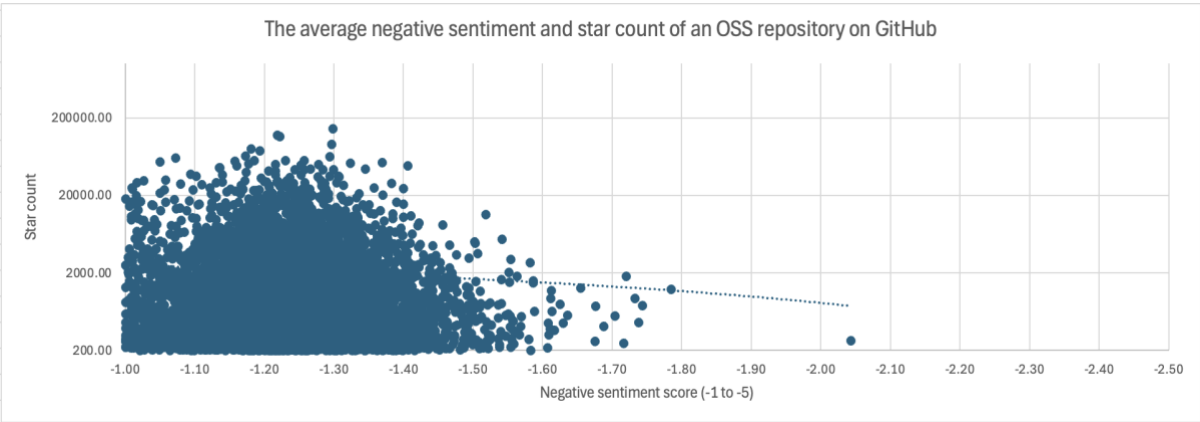

Figure 4b. Source: author. "The average negative sentiment and star count of an OSS repository." Apr. 2025.

This suggests that as the average negative sentiment for a repository increases the star count for the repository decreases. The p-value for the correlation is 0.007 making it extremely unlikely to have happened by chance alone. The null hypothesis is therefore rejected, establishing a weak negative correlation between the average negative sentiment of the comments on a repository and the star count of the repository. However, similarly to the positive sentiment, the low p-value is more due to the large sample size as opposed to an actual correlation between the two variables. In reality, a correlation coefficient of -0.003 is far too small to be practical in any way.

The null hypothesis for RQ1 is statistically rejected due to the low p-values, but the correlation is far too small to prove useful in practice. Ultimately, neither average positive nor average negative sentiment has a role in determining repository success.

Research question 2

To investigate the impact that the popularity of the programming language has on the popularity of the OSS project, I constructed Table 3, ordered by the total number of repositories for that language.

Programming Language	Stack Overflow	Stars (total)	Forks (total)	Average Stars	Median Stars	Count
Python	51.0%	27,319,316	4,775,757	1,403.37	465	19,467
JavaScript	62.3%	22,314,613	3,833,472	1,739.93	503	12,825
TypeScript	38.5%	19,674,906	2,676,492	2,113.54	541	9,309
Go	13.5%	14,105,959	1,763,904	2,043.75	564	6,902
Java	30.3%	10,149,228	2,585,055	1,551.40	526	6,542
C++	23.0%	9,580,471	1,815,257	1,631.83	507	5,871
C#	27.1%	5,131,232	932,721	1,292.83	477	3,969
C	20.3%	5,139,941	936,213	1,415.18	524	3,632
PHP	18.2%	3,953,076	769,204	1,165.41	458	3,392
Rust	12.6%	6,602,266	502,209	1,968.48	553	3,354
Shell	33.9%	3,855,362	715,598	1,481.69	492	2,602
Swift	4.70%	3,385,866	357,148	1,313.37	535	2,578
Kotlin	9.4%	2,522,290	361,780	1,170.44	480	2,155

Table 3. Source: author. "Programming Language Popularity and GitHub Statistics Comparison." Apr. 2025; Stack Overflow, "2024 Stack Overflow Developer Survey."

The top 10 programming languages, by the count of total repositories using it, are Python, JavaScript, TypeScript, Go, Java, C++, C#, C, PHP, Rust, Shell, Swift, and Kotlin.

The list is similar to the top programming languages found by Sanatinia and Noubir in 2016, Altherwi in 2021, and Milicin in 2024, but once again has some discrepancies. Most notably, JavaScript was consistently found to be the most popular programming language, however, from my findings, it has been replaced by Python (which was usually the second most popular). Some high developer use percentages as determined by Stack Overflow with a low total count can be explained by the language use cases. For example, Shell wasn't designed for large-scale projects and is more often used to make smaller automation scripts. As such, it has a low total count of repositories despite its relatively frequent use by developers.

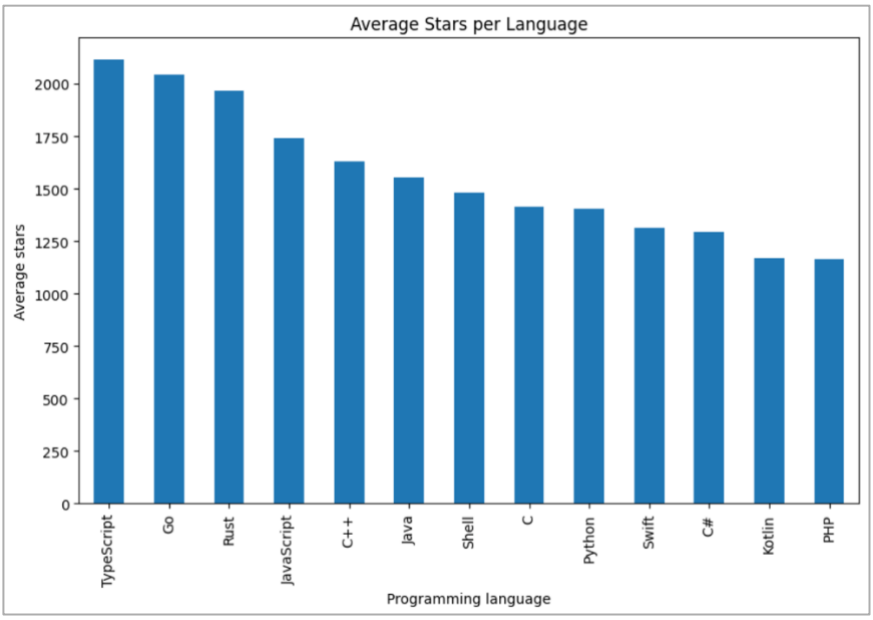

Figure 5. Source: author. "Average Stars per Programming Language on GitHub." Apr. 2025.

As highlighted in Figure 5, the programming language of an OSS project influences the total number of stars. The most drastic difference can be seen when comparing TypeScript and PHP with a difference of 948.13 stars. This aligns with Milicin's findings which found that the programming language of a repository does affect the star count. However, a further investigation will be made by comparing the popularity of the language itself to the popularity of the repositories using that language. To do that, I constructed Figure 6 where

the star count of the repository is on the y-axis and the popularity of the programming language is on the x-axis.

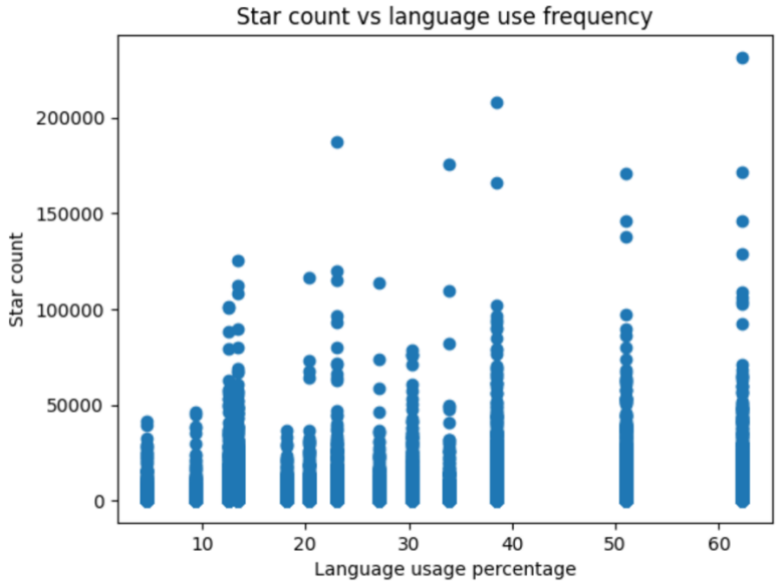

Figure 6. Source: author. "Correlation Between Programming Language Usage Frequency and Repository Star Count." Apr. 2025; Stack Overflow, "2024 Stack Overflow Developer Survey."

The Pearson correlation coefficient was calculated to be -0.00012, indicating an extremely weak negative correlation. However, this correlation is far too low to bear any real-world significance. This is further emphasized by the extremely high p-value of 0.9726.

The null hypothosis for RQ2 could not be rejected as the p-value is greater than 0.01. Therefore, I couldn't establish the programming language's popularity as the reason for the popularity of an OSS repository. However, despite the low p-value, the low calculated correlation suggests that there is no relationship between the OSS repository's popularity and the popularity of the programming language that it uses.

Conclusion

Through this analysis, I investigated the role of two aspects of OSS in determining its success: the sentimentality of the discussions and the popularity of the programming language used by the OSS. For the first research question (RQ1), although the results were statistically significant and rejected the null hypotheses, the correlation coefficients for both

the positive and negative sentiment to the star count of an OSS repository were extremely weak. This suggests that the overall positive or negative sentiment has practically no effect on the repository's success. Nevertheless, the findings that comments tend to be more positive rather than negative not only align with prior research done by Assavakamhaenghan but also support Szatmáry P.'s reasoning by suggesting that comments are generally welcoming to newcomers.

For the second research question (RQ2), despite finding that programming languages do influence the star count which is in line with Milicin's research, the popularity of the programming language itself is not the reason. My research also finds further changes in the topmost popular programming languages, most notably Python overtaking JavaScript at the number one spot. Despite failing to reject the null hypothesis, the calculated correlation was so low, that it is unlikely that any relationship between the popularity of an OSS repository's programming language and the star count exists. My research therefore indicates no correlation between the popularity of an OSS repository's programming language and the popularity of the OSS.

In short, neither sentimentality nor language popularity can explain why some OSS repositories become more popular than others.

Limitations and future considerations

Although my method was very extensive, certain limiting factors may have influenced the data. First, using the common topic tags among non-OSS repositories to filter for only OSS repositories may have introduced certain bias away from certain topics such as education-focused OSS as many of the blacklisted topic tags were related to learning (e.g. "education" and "tips"). While my large sampling certainly lowered the impact that this bias has, it is important to keep that in mind. Another limitation to consider is the accuracy of SentiStrength-SE. While SentiStrength-SE provides the best accuracy for its great efficiency,

it certainly didn't give completely accurate scores for how positive or negative humans would interpret a comment.

For future research, a case study for determining the influence of sentimentality could result in more accurate results. A case study could enable researchers to take the time to analyze a broader range of emotions such as joy, anger, or sadness rather than only positive or negative sentiment. One factor that I considered only in my literary review, but didn't include in my method was the fact that the popularity of a programming language quickly changes over time. An analysis of the influence of the popularity of a programming language on the star count over time could establish it as an indicator of star count. Stack Overflow Developer Survey 2024 does have data for the popularity of a programming language from every year and there are some event-based datasets for GitHub providing timestamps for when each star was given.

Implications for developers

There are many aspects that developers or maintainers of OSS must consider when attempting to grow their OSS repository and thereby community. My research sheds light on two aspects of OSS: the sentimentality of discussion and the influence of the programming language used. My research finds that developers should not prioritize trying to increase the positive sentiment or decreasing the negative sentiment in discussions, as neither had a relationship to the OSS repository's popularity or star count. Similarly, while choosing a popular programming language can offer certain benefits, such as more libraries, resources, and contributors, my results show that the popularity of a programming language does not influence the popularity of the OSS repository itself. Developers should therefore prioritize other factors when attempting to grow their OSS community.

Appendix A

Sample code used for collecting repository data.

```
45    let ids = 0;
46
47    while (high >= 200) {
48            let res = await octokit.request('GET /search/repositories', {
49                    headers: {
50                      'X-GitHub-Api-Version': '2022-11-28'
51                    },
52                    q: `stars:${low}..${high}`,
53                    per_page: 100,
54                    page: 1,
55                    sort: "stars",
56                    order: "desc"
57            });
58
59            console.log(`Star range: ${low}..${high} (${res.data.total_count})`);
60
61            if (res.data.total_count > 1000) {
62                    low += Math.ceil((high - low) / 2);
63                    continue;
64            }
65
66            console.log("Found!");
67
68            let total = res.data.total_count;
69
70            let time;
71            for (let i = 1; i <= Math.ceil(total / 100); i++) {
72                    time = Date.now();
73
74                    let reposRes = await octokit.request('GET /search/repositories', {
75                            headers: {
76                              'X-GitHub-Api-Version': '2022-11-28'
77                            },
78                            q: `stars:${low}..${high}`,
79                            per_page: 100,
```

Full code available online at https://github.com/BoogeyMan24/ap-research/blob/main/src/repos.js.

Appendix B

Sample code used for collecting comment data.

```
let comments = [];
let commentsData = resInit.data;

commentTotal = commentsData.length;

gettingComments = true;

progress();

for (let i = 0; i < commentsData.length; i++) {

        let commentData = commentsData[i];

        let comment = [
                row.id,
                row.name,
                commentData.id,
                commentData.node_id,
                commentData.html_url,
                commentData.author_association,
                commentData.user.type,
                commentData.body
        ];

        if (commentData.user.type == "Bot") {
                commentBots++;
        }

        comments.push(serializeRow(comment) + "\n");
        commentCount++;
}
```

Full code available online at https://github.com/BoogeyMan24/ap-research/blob/main/src/comments.js.

Works Cited

Ahmed, Toufique, et al. "SentiCR: A customized sentiment analysis tool for code review interactions." *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Oct. 2017, pp. 106–111, https://doi.org/10.1109/ase.2017.8115623.

Altherwi, Muna. "An Empirical Study of Programming Languages in Open-Source Software Projects Based on Mining Software Repositories." *ePrints Soton*, University of Southampton, 27 Aug. 2021, eprints.soton.ac.uk/456713/.

Assavakamhaenghan, Noppadol, et al. "Does the first response matter for future contributions? A study of first contributions." *Empirical Software Engineering*, vol. 28, no. 3, May 2023, https://doi.org/10.1007/s10664-023-10299-7.

Borges, Hudson, and Valente, Marco Tulio. "What's in a github star? understanding repository starring practices in a social coding platform." *Journal of Systems and Software*, vol. 146, Dec. 2018, pp. 112–129, https://doi.org/10.1016/j.jss.2018.09.016.

Calefato, Fabio, et al. "EmoTxt: A toolkit for emotion recognition from text." 2017 Seventh International Conference on Affective Computing and Intelligent Interaction Workshops and Demos (ACIIW), Oct. 2017, pp. 79–80, https://doi.org/10.1109/aciiw.2017.8272591.

Chesbrough, Henry. "Measuring the Economic Value of Open Source." *The Linux Foundation*, Mar. 2023, www.linuxfoundation.org/research/measuring-economic-value-of-os.

Finley, Klint. “Whoa. Microsoft Is Using Linux to Run Its Cloud.” *Wired*, Wired, 18 Sept. 2015, www.wired.com/2015/09/microsoft-using-linux-run-cloud/.

GitHub. “Search stars:>=0.” GitHub, github.com/search?q=stars%3A%3E%3D0&type=repositories. Accessed Apr. 2025.

Hoffmann, Manuel, et al. “The Value of Open Source Software.” *SSRN*, 16 Jan. 2024, ssrn.com/abstract=4693148.

Islam, Md Rakibul, and Zibran, Minhaz F. “Sentistrength-SE: Exploiting domain specificity for improved sentiment analysis in software engineering text.” Journal of Systems and Software, vol. 145, Nov. 2018, pp. 125–146, https://doi.org/10.1016/j.jss.2018.08.030.

JetBrains. “Software Developers Statistics 2024 - State of Developer Ecosystem Report.” *JetBrains*, JetBrains, June 2024, www.jetbrains.com/lp/devecosystem-2024/.

Milicin, Andrei-Radu. “Investigating the impact of GitHub repository features on popularity.” *Radboud University Nijmegen,* June 2024, www.cs.ru.nl/bachelors-theses/2024/Andrei-Radu_Milicin___1076192___Investigating_the_impact_of_GitHub_repository_features_on_popularity.pdf.

Sanatinia, Amirali, and Noubir, Guevara. “On GitHub’s Programming Languages.” *ResearchGate*, Mar. 2016, www.researchgate.net/publication/297493968_On_GitHub’s_Programming_Languages.

Sarker, Jaydeb, et al. “A benchmark study of the contemporary toxicity detectors on software engineering interactions.” 2020 27th Asia-Pacific Software Engineering Conference (APSEC), Dec. 2020, pp. 218–227, https://doi.org/10.1109/apsec51365.2020.00030.

Shimada, Naomichi. “An Exploratory Study of Donations to Individual Developers via GitHub Sponsors.” *Nara Institute of Science and Technology*, Mar. 2022, naist.repo.nii.ac.jp/record/11810/files/R017831.pdf.

Stack Overflow. “2024 Stack Overflow Developer Survey.” *Stack Overflow*, Stack Overflow, June 2024, survey.stackoverflow.co/2024/.

Szatmáry, Péter, and Schweitzer, Maciej Karol. “Exploring the Impact of Open-Source Software Documentation on Contributors’ Motivation and Participation.” *Aalborg Universitet*, 8 June 2023, projekter.aau.dk/projekter/files/536282994/P10_fin_202306081409.pdf.

Zhou, Jiayuan, et al. “Studying donations and their expenses in open source projects: A case study of github projects collecting donations through open collectives.” *Empirical Software Engineering*, vol. 27, no. 1, 30 Nov. 2021, https://doi.org/10.1007/s10664-021-10060-y.